Name & Surname **Polimi ID**

Exercise 1

A. Describe the scan followed by a reduce parallel pattern providing PRAM complexity and show at least one example discussed during the lectures. (1)

B. Describe the parallel pipeline pattern and one condition required for good scalability. (1)

C. Given the following C-based code:

```
for (i=1; i<n; i++)
    for (j=1; j<m; j++)
        x[i][j] = x[i-1][j] + x[i][j-1] + y[i][j]

for (i = 0; i < 100; i++) {
    for (j = 1; j < 100; j++) {
        z[i][j] = z[i][j-1] - x[i][j-1] + w[i][j-1];
    }
}
```

where matrices x, y, z, and w store integers and already contain valid data before the execution of the code.
Discuss when function g prevents code parallelization (a practical example is required). Is the memory layout of x,y,z and w creating issues for the parallelization of the code (show an example where this is going to happen)? How and under what conditions can the previous code be parallelized? (2)

Name & Surname **Polimi ID**

Exercise 2

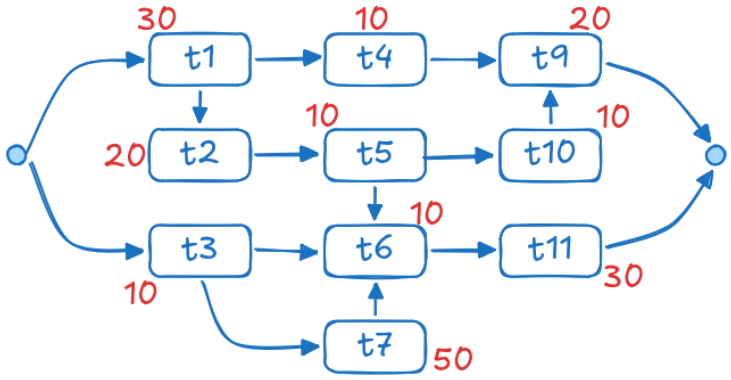
A. Describe the Pack pattern and give its PRAM work and time complexity. (2)

B. A program has three independent, known-at-compile-time stages, followed by a recursive computation that creates an unknown number of irregular subproblems. For each phase, choose between OpenMP sections and tasks, and justify the choice in terms of scheduling, load balancing, and synchronization. Mention the OpenMP construct needed to ensure that all generated work is complete before the program continues. (2)

Name & Surname **Polimi ID**

Exercise 3

A. Consider the following task graph:



Calculate Work and Parallelism. Imagine having the possibility to optimize a task, halving its computational cost. Which one would you choose to reduce the span? (motivate) (2)

B. Within a `parallel` OpenMP section, there is only a plain `for` loop. What would happen if we didn't use the OpenMP `for` construct? (1)

C. State the pros and cons of dynamic and static schedules in OpenMP when applied to the `for` worksharing construct. (1)

Name & Surname **Polimi ID**

Exercise 4

A. Consider the following snippet of code of an MPI program with three processes:

```
int x = 10;
if (rank == 0) {
      MPI_Recv(&x, 1, MPI_INT, 2, 2, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
      MPI_Recv(&x, 1, MPI_INT, 1, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
} else if (rank == 1) { MPI_Send(&x, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
} else if (rank == 2) { MPI_Send(&x, 1, MPI_INT, 0, 2, MPI_COMM_WORLD); }
```

Is it possible that it leads to a deadlock? (Explain) (1) **This question can be skipped if you got 1 or 3 points in the second challenge.**

note: in both MPI functions, the 4th parameter is the rank, while the 5th is the tag)

B. Which is a scenario where it is useful to use non-blocking semantics? (1)

C. What `MPI_Comm_split` does? Describe an application scenario where its usage is useful (2) **This question can be skipped if you got 2 or 3 points in the second challenge.**